EX02 - Ideal Rocket Equation - Imports and Logarithms

Beginner Step-by-Step Guide

This guide follows the real file `ex02_rocket_equation.py`. It explains the problem first and then the code from the perspective of someone starting Python.

1. What we want to solve

Calculate Delta_v from Isp, g0, initial mass m0 and final mass mf.

Problem variables / elements

Name	Meaning
Isp	specific impulse [s]
g0	reference gravity [m/s^2]
m0	initial mass [kg]
mf	final mass [kg]
Delta_v	ideal velocity increment [m/s]

Equation or logic on paper

```
Delta_v = Isp * g0 * ln(m0 / mf)
```

Manual check

Control case: Isp=250, g0=9.81, m0=500000, mf=250000. Mass ratio is 2 and ln(2)~0.6931, giving Delta_v~1699.7 m/s. The current file uses mf=500000, so the m0<=mf validation branch runs.

2. What we already know and what is new

Already seen:

- variables
- if / elif / else
- print()

New in this exercise:

- import numpy as np
- calling np.log()
- intermediate variable mass_ratio
- several chained validations

3. New Python concepts

1. Import a library

Syntax: `import numpy as np`

import loads a module. as np creates a short alias so we can write np.log().

2. Call a function

Syntax: `np.log(mass_ratio)`

Parentheses mean log is executed with mass_ratio as its argument. np.log returns the natural logarithm.

3. Decision chain

Syntax: `if ... elif ... else:`

Python checks branches from top to bottom. Once the first True condition is found, that branch executes and the rest are skipped.

4. Code step by step

Step 1. Import NumPy

```
import numpy as np
```

We need a mathematical function outside basic syntax: np.log().

Step 2. Define inputs

```
Isp = 250
g0 = 9.81
m0 = 500000
mf = 500000
```

Each variable represents one model quantity.

Step 3. Validate before calculating

```
if Isp < 0:
    ...
elif m0 <= 0:
    ...
elif mf <= 0:
    ...
elif m0 <= mf:
    ...
```

<= means less than or equal to. The exercise prevents calculation when an input violates the defined rules.

Step 4. Calculate mass ratio

```
mass_ratio = m0 / mf
```

An intermediate variable exposes an important part of the equation and makes debugging easier.

Step 5. Apply the logarithm

```
delta_v = Isp * g0 * np.log(mass_ratio)
```

This line is the direct translation of the mathematical equation.

5. Execution flow

```
inputs
   -> validation chain
   -> mass_ratio
   -> np.log(mass_ratio)
   -> delta_v
   -> print
```

6. Expected result / check

With the file as provided, m0==mf and the validation message is printed. To test the calculation, change mf to 250000 and expect ~1699.7 m/s.

7. Common beginner mistakes

- Confusing np.log with base-10 log.
- Calculating before validation.
- Not understanding only the first True branch runs.
- Forgetting / performs real division.

8. If you remember only five things

- import loads external tools.
- np is an alias.
- function(...) executes a function.
- elif orders validations.
- Intermediate variables make code easier to read.

9. Mini-practice

Change one input. Before running, write a comment predicting what should change and why. Then run the file and compare your prediction with the output.

Completion criterion

Move on when you can explain the problem, equation/logic, new Python concepts and execution flow without simply reading the code.